When to Retrain: Label-Free, Cost-Aware Drift Triage for Encrypted Traffic Classifiers

Lavnish J.
Reg. No. 24BCE2127

Darsh S.
Reg. No. 24BDS0082

Dhruv M.
Reg. No. 24BKT0116

***Abstract*—A deployed encrypted-traffic classifier degrades as traffic drifts, yet ground-truth labels are unavailable after deployment and every retrain costs compute and labeling effort. We present a label-free, cost-aware triage layer that decides when a frozen classifier should be retrained. It fuses eight signals computed only from the classifier's probability output and raw input features into one drift score, removes weekly seasonality, detects persistent shifts with PELT, and prices the expected accuracy loss against the cost of retraining. A runtime-guarded label store refuses label access outside calibration and retraining. On a year of CESNET-TLS-Year22 traffic (50 services, five seeds) the score tracks true macro-F1 loss (Spearman 0.58; 0.85 after detrending). Our cumulative-cost rule reaches 0.842 macro-F1 with 5.4 retrains, against 0.709 for never retraining and 0.831 for the myopic rule, and lies 0.26 F1 points below the hindsight-optimal schedule at the same budget (mean over 5 seeds). On CESNET-QUIC22 it retrains on the day of the documented certificate change in all five seeds, while a fixed alarm threshold never fires.**

***Index Terms*—encrypted traffic classification, concept drift, retraining policy, change-point detection, label-free monitoring**

I. Introduction

Operators classify encrypted traffic from what stays observable: packet sizes, timing, direction and flow-level statistics [1], [2], [3]. Models built on these features work well in the week they are trained. They do not stay accurate. Traffic drifts as applications update, certificates rotate and exporters change; in a month-long study of ISP QUIC traffic, classifiers trained on one week lost 9–13 accuracy points within three weeks [4]. Restoring accuracy means retraining, which costs compute and, more importantly, fresh labels. For encrypted traffic the labels are the scarce resource. QUIC and Encrypted ClientHello remove the plaintext handshake fields earlier work used to label flows [4], [5], so an operator cannot cheaply grade a deployed model.

Two schedules dominate practice. Periodic retraining ignores the state of the traffic and spends labels whether or not the model has degraded; a fixed alarm threshold ignores what retraining costs [6]. Cost-optimal retraining has been formalized as a Markov decision process [7], but the detector in that formulation reads labels. The encrypted-traffic studies that measure drift [8], [9], [10] stop short of saying when a measured shift justifies retraining, and Rachmawati et al. note the need for continuous updating while leaving its scheduling open [3].

We close this gap with a triage layer that sits beside a frozen classifier and decides, once per week and without labels, whether retraining is worth its price. Our contributions are:

- A *label-free observer* that fuses eight signals from the classifier's probability output and raw inputs into one drift score, using sign orientation, family-balanced weights and per-generation baselines (Sec. IV-A).
- A *cost-aware decision layer*: a frozen isotonic map from score to expected macro-F1 loss, a myopic rule that compares one week's expected loss with the fixed cost of a retrain, and a cumulative-cost rule that we introduce because the myopic rule cannot price a persistent, moderate loss (Sec. IV-C).
- An *evaluation harness* with isolated policy timelines, an exact dynamic-programming bound on what any schedule achieves with hindsight, and three-layer enforcement that the decision path never reads labels (Secs. IV-D and V).
- A *full-year, five-seed evaluation* on CESNET-TLS-Year22 and a cross-protocol check on CESNET-QUIC22, with negative results reported next to positive ones (Sec. VI).

II. Related Work

A. Encrypted traffic classification

Port-based identification collapsed once applications adopted dynamic ports, and payload inspection is expensive, privacy-invasive and impossible on encrypted flows [1]. Machine learning on flow statistics and on packet size and timing sequences replaced it: decision trees on bidirectional flow features classify social-media applications [2], and deep models followed [3]. Contextual TLS metadata identifies malware without decryption [11], while recent accuracy work centers on learned representations: ET-BERT pre-trains transformers on datagrams [12], TFE-GNN fuses temporal graph structure [13], and MsETC learns multi-scale contrastive representations [14]. Encryption across TLS, HTTP/2 and QUIC changes which traffic characteristics an operator can observe at all [15]. QUIC hides the handshake SNI, and recovering ground truth would need decryption infrastructure that operators judge infeasible at provider scale [4]; neural classifiers degrade sharply once Encrypted ClientHello conceals the ClientHello [5]. This body of work improves the classifier. It does not say when a deployed one has stopped being trustworthy.

B. Drift in traffic classification

Drift in this domain is rapid, seasonal and class-dependent. Classifiers trained on one QUIC week lose 9–13 accuracy points by the third week, traced to a provider certificate change that altered early packet sizes [4]. Anderson and McGrew show that distribution changes strongly affect malware classifiers trained on historical data [16]. Malekghaini et al. measure drift across more than two years of ISP TLS data and find that the damage depends on the feature representation and the granularity of the task [10]. Jančička et al. confirm a weekly seasonal component over a year of TLS traffic and observe that the features a classifier relies on most are disproportionately those that shift; they leave open how much change counts as meaningful [9]. Kurapov et al. conclude that monthly retraining suffices, but their data could not locate the point at which quality starts to degrade [8]. These two open questions are what a retraining trigger must answer.

C. Label-free drift detection

Jensen–Shannon divergence between traffic windows detects concept drift without labels and allows an earlier classifier to be reused [17]. Lughofer et al. track classifier confidence with a modified Page–Hinkley test [18]; Sethi and Kantardzic count samples that land in the classifier's margin of uncertainty and cut false alarms relative to detectors that watch only features [19]; Souza et al. detect drift efficiently on high-speed streams with delayed or missing labels [20]. For change points, Aminikhanghahi et al. propose an unsupervised density-ratio detector with short delay [21], and Smirnov shows that the penalty of PELT is a real optimization problem: small values raise false alarms, large values suppress detection [22]. These methods answer "did something change?" Whether the change justifies an expensive action lies outside their scope.

D. Retraining policies and adaptation

Gudepu et al. argue that fixed-threshold and periodic retraining are both inadequate and trigger retraining with an unsupervised outlier detector [6]. Kanda et al. model retraining under drift as a Markov decision process with explicit retraining and error costs and find detector-based policies cheaper than periodic ones, although their detector uses labels [7]. Bagui et al. connect an Isolation Forest drift detector to incremental or full retraining on network-security data [23]. Liu et al. ask for labels only on the most informative samples in an online active-learning framework [24], and Cai et al. update encrypted-traffic classifiers incrementally with contrastive prototypes [25]; both still consume labels. LEAF shows in cellular networks that retraining can itself reduce accuracy and mitigates this with selective forgetting and oversampling [26]. Li et al. match the adaptation strategy to the drift type [27], and Gama et al. survey the field [28].

E. Gap

Encrypted-traffic studies establish that drift occurs [4], [8], [9], [10]. Label-free detectors show that it can be observed without labels [17], [18], [19], [20]. Adaptive retraining beats static models [6], [7], [23], [24], [25]. What is missing is the decision between them: a label-free trigger, evaluated on encrypted traffic, that prices expected loss against retraining cost and is judged against the best schedule achievable with hindsight.

III. Problem Setting

A classifier f_θ is trained on labeled flows from an initial window W0 (weeks 1–2) and then frozen. Flows arrive in weekly batches $t = 3, \dots, 52$. At the end of week t a policy returns *retrain* or *hold*; a retrain fits a new model on labels from the trailing $R = 2$ weeks and deploys it for week $t+1$. The monitor sees only the probability matrix $P_t = f_\theta(x_t) \in \mathbb{R}^{n\times 50}$ and the raw features x_t. Labels are read in exactly two places: once, for the window that calibrates the loss map g, and at the moment a retrain fires. A policy is judged by mean weekly macro-F1, retrain count and cost.

IV. Method

A. Label-free observer

For each day we compute eight signals from a sample of 10,000 flows: the mean and 10th-percentile maximum probability, the mean margin between the top two classes and the mean entropy of P; the Jensen–Shannon divergence between the predicted-class histogram and the reference histogram of the deployed model; the mean per-feature Kolmogorov–Smirnov statistic and an unbiased Gaussian-kernel MMD^2 [29] against a fixed reference sample (median-heuristic bandwidth); and the reject rate, the fraction of flows whose maximum probability falls below the 10th percentile of the reference.

Each signal enters with an orientation $\sigma_i \in \{-1, +1\}$ so that degradation always raises the score: maximum probability and margin fall under drift, while entropy, the divergences and the reject rate rise. Signals are centered on a per-generation baseline $b_i^{(m)}$, their value on a held-out fifth of generation m's own training window at the daily sample size, and scaled by $\hat{\sigma}_i$, the robust (MAD) day-to-day spread of $s_i - b_i$ for the initial model over weeks 3–8, frozen thereafter. The composite is

$$D_t = \sum_i w_i \, \sigma_i \, \frac{s_i(t) - b_i^{(m)}}{\hat{\sigma}_i}. \tag{1}$$

The hand-set weights give each of four families (confidence, prediction divergence, input drift, reject rate) a total weight of $1/4$; equal per-signal weights would hand the four collinear confidence statistics half the vote. Per-generation baselines keep D comparable across retrains: it measures how far a model's inputs and outputs have moved from where they stood when that model was trained, in units of normal daily variation. Nothing in the score is learned.

B. Conditioning and change detection

The daily series of D (eight missing calendar days are linearly interpolated) is deseasonalized with robust STL at period 7 [30], keeping trend and residual; the day-of-week cycle is documented for this dataset [9]. We normalize with the

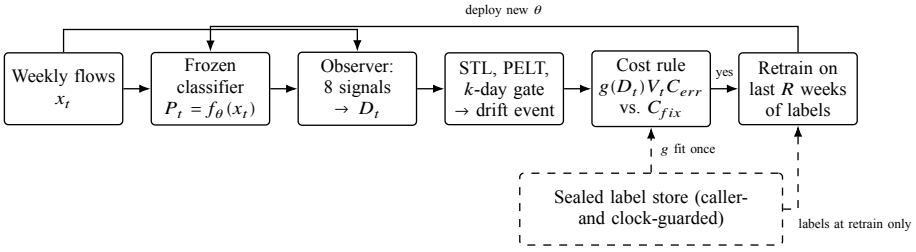


Fig. 1. System overview. Everything left of the dashed store is label-free; labels flow only into the one-time calibration of g and into a fired retrain.

median and MAD of the calm window (weeks 3–8) rather than a trailing window, because a trailing median acts as a high-pass filter and cancels a persistent step after half its length. PELT with an ℓ_2 cost and minimum segment length 3 [31] finds change points within the current model's segment. The penalty is the smallest value on a logarithmic grid that yields no change point on the calm window ($\lambda = 8$ here); the full sweep is logged, since the penalty trades false alarms against missed detections [22]. A change point becomes a *drift event* only if the shift is upward, the next $k = 3$ days stay above the pre-change mean, and the shift is still active today. The chain runs on the history seen so far, and a test that perturbs future observations confirms that past detections do not change.

C. Cost-aware decision

Let g map D to expected macro-F1 loss. We fit g once by isotonic regression on the days of weeks 3–10, using the true loss of the initial model (the only labeled reading outside retraining), and freeze it. Beyond the largest calibration score, g continues with the non-negative least-squares slope instead of staying flat: a flat extension prices any drift larger than the calibration window at the window's maximum, and in a synthetic stress test it delayed the cumulative rule's first retrain from week 14 to week 17 after a shift at week 12. We did not run the flat variant on the real data. Let V_t be the week's flow volume, C_{err} the cost of one flow of macro-F1 loss, and $C_{fix} = C_{retrain} + C_{label}$. The *myopic rule* (P4) retrains iff a drift event is active and

$$g(D_t)\, V_t\, C_{err} > C_{fix}. \tag{2}$$

This compares a per-week loss rate with a one-off cost, so a persistent, moderate loss that stays just under the threshold is never corrected even though its cost grows without bound. The *cumulative rule* (P4c) accumulates the excess loss since the current model was deployed,

$$A_t = \sum_{u=u_0}^{t} \max(0,\, g(D_u) - g_0)\, V_u\, C_{err}, \tag{3}$$

retrains once $A_t > C_{fix}$ and resets. The floor g_0 is the loss measured in the first week after the initial deployment (week 3), which no retrain removes because every model is deployed one week after its training data. Only the ratio $r = C_{err}V_{ref}/C_{fix}$ matters for decisions, with V_{ref} the mean weekly volume in W0. It is the one operator-facing knob, and we sweep it over three orders of magnitude.

D. Label isolation

The label-free claim holds only if no label can reach the decision path, so we enforce it instead of relying on discipline. First, a static import-graph test fails if any module in the observer, detector, decision, classifier or replay packages reaches the label-bearing loader, harness or label store, directly or transitively and in every import form (`from pkg import module`, relative and `importlib`); the checker is itself mutation-tested against nine evasion patterns, and the form `from pkg import module` slips past a test that inspects only the module name of the import node. Second, the store checks its caller at runtime and refuses labels for any week after a simulation clock that only moves forward. Third, a behavioral test replaces the grader's output with random counts and verifies that the retrain decisions of P3, P4 and P4c stay identical while the oracle's change. Only the oracle P5, a non-competing upper bound, uses true performance.

V. Experimental Setup

A. Data

CESNET-TLS-Year22 [32] holds backbone TLS flows for all of 2022. We stream the public archive and keep a uniform Bernoulli sample of about 100,000 flows per day (the rate is stored, so true volumes are recoverable), then cap each replay day at 10,000 flows. Sampling is label-agnostic: a per-service cap would read labels on the operational stream and flatten the class prior that the divergence signal watches. The 50 services with the largest volume in W0 form a closed world; ranking by whole-year volume would use future information. Features are 60 flow statistics (bytes, packets and duration per direction, packet-sequence summaries, TCP flag counts, flow-end reasons and four 8-bin packet-size and inter-arrival histograms), log-transformed on count columns and standardized with a scaler fitted once on W0. The exporter update in week 10 changes packet-sequence data [32]. Replay covers weeks 3–52, 50 decisions.

CESNET-QUIC22 [33] spans 28 days, too few for weekly decisions, so we decide daily: W0 is days 1–7, replay covers days 8–28, STL is disabled and a retrain uses the last 7 days. The documented certificate change falls on day 22, in week 47 [4].

B. Classifier, policies and metrics

The substrate is a 100-round, 31-leaf LightGBM [34] with learning rate 0.1. With library defaults, 50-class training di-

verged on W0 (held-out accuracy 0.16 at a mean maximum probability of 0.9999); a hessian floor and L2 regularization, chosen on W0 alone, fix it (accuracy 0.845), and the trainer now fails loudly on that signature. A retrain fits at most 100,000 flows. We compare P0 never retraining; P1 and P2, periodic every 4 and 1 weeks; P3, an alarm when D exceeds the calm median by three robust standard deviations; P4; P4c; and the oracle P5, which retrains when the deployed model's true loss exceeds 0.05. Each family's knob is swept: periodic $k \in \{1, 2, 3, 4, 6, 8, 13, 26\}$, the P3 threshold, $r \in \{0.3, \ldots, 1000\}$ for P4 and P4c, and the P5 target. Costs are $C_{retrain} = 100$ and $C_{label} = 400$ in illustrative units; the default is $r = 10$.

We report mean weekly macro-F1 over the 50 replay weeks and the retrain count. Cost is $[\sum_t \max(0, F_{ref} - F_t) V_t C_{err}]/C_{fix} + n_{retrain}$ in units of one retrain, where F_{ref} is the initial model's held-out F1; clipping at zero matters, because an unclipped sum credits weeks above the reference and turns negative at large r. Detection delay counts weeks from the first week the deployed model's true loss exceeds 0.03 to the next retrain, and a retrain is a false trigger if the loss in its week is below 0.03. Because a retrained model depends only on its training week, the macro-F1 of every (generation, week) pair can be tabulated once, 1,275 pairs per seed, and dynamic programming then yields, for each retrain budget, the schedule with the highest mean F1 and, for each r, the schedule of minimum cost. We report each policy's *gap*: that frontier at the policy's own budget and seed minus the policy's F1. Five seeds vary classifier randomness and subsampling, not the underlying traffic.

VI. Results

A. Drift, and whether D tracks it

Never retraining (P0) falls from a held-out F1 of 0.840 to a weekly mean of 0.709, and to 0.610 by week 52. The decline has two phases: a step at the week-10 exporter update (weekly F1 0.823 in week 9, 0.784 in week 10, 0.757 in week 11), then slow erosion with a second drop after week 44. Antivirus services suffer most; `bitdefender-gravityzone` loses 0.63 F1 and `avast` 0.52, against a median loss of 0.15 across the 50 services.

The label-free score tracks this loss. Across 50 weeks the Spearman correlation between weekly D and true F1 loss is 0.58 (Pearson 0.86); after removing the linear trend from both series it is 0.85 (Pearson 0.94), and week-over-week changes correlate at 0.80. All clear the 0.3 gate we fixed before building the cost layer. A check over only the first 11 weeks, run while data was still downloading, gave 0.82; the full-year value is lower because D stays high through the second half of the year while loss keeps growing. The signals are not equal (Table II). The confidence family and the reject rate carry the correlation (Spearman 0.67–0.73), JS divergence of predicted classes is nearly uninformative (0.05), and input drift is weak on ranks (KS 0.20, MMD 0.17). The composite correlates less than the reject rate alone (0.58 against 0.73) because the three weak signals dilute it. We did not ablate signals in the decision rules, so we cannot say whether a confidence-only score would also decide better.

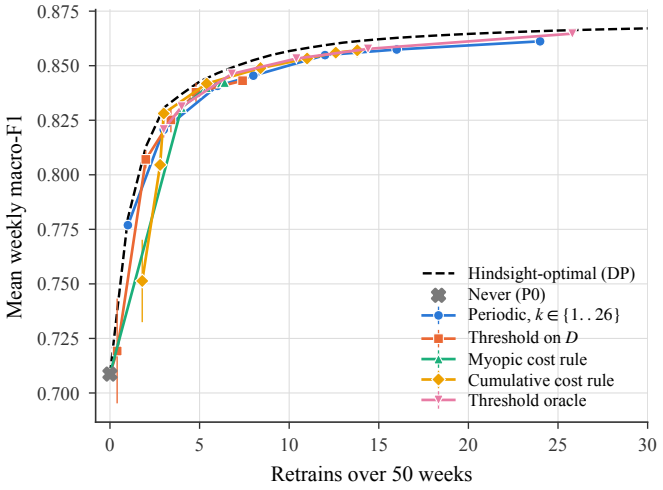


Fig. 2. Mean weekly macro-F1 against retrains for each policy family (markers: mean over five seeds; bars: standard deviation) and the DP-optimal frontier (dashed; mean over 5 seeds). Curves for P3, P4, P4c and P5 are traced by sweeping their knobs.

B. Policy comparison

Table I gives the default operating point. P4c retrains 5.4 times and lifts macro-F1 to 0.842; the myopic rule P4 retrains 4 times and reaches 0.831. P4c sits 0.26 points below the DP-optimal schedule at its budget and P4 0.61 (mean over 5 seeds), and P4c reacts in 2.1 weeks where P4 needs 3.2. Both fire in week 10, the exporter update, in all five seeds. The threshold rule P3 raises no false trigger but reacts 5.8 weeks late on average and ends 0.80 points below the frontier. Retraining every four weeks (P1) reaches 0.855 with 12 retrains and weekly retraining (P2) 0.868 with 49, at false-trigger rates of 0.50 and 0.92. The oracle P5, which reads true performance, sits 0.30 points below the frontier: even an oracle with a fixed loss target is not optimal.

Against calendars the result is modest. At matched budgets the label-free rules and tuned periodic schedules land within a few tenths of a point of each other. Periodic retraining every 8 weeks uses 6 retrains and reaches 0.841, 0.58 points below the frontier; P4c uses 5.4 and reaches 0.842, 0.26 below it. Near twelve retrains, retraining every four weeks (0.855, gap 0.46) and P4c at $r = 100$ (0.853, gap 0.46) are indistinguishable. The periodic interval is chosen in hindsight on the evaluation year, whereas P4c at $r = 10$ is not tuned on it, and of the policies in Table I P4c has the lowest cost, below the oracle P5. What the triage layer adds is timing that follows the traffic (Sec. VI-E), a shorter detection delay than the 8-week calendar (2.1 against 2.7 weeks), and a cost knob an operator can set without knowing the right interval in advance.

C. Distance from the optimal schedule

Fig. 2 places every family against the DP frontier (mean over 5 seeds). The frontier rises steeply to 0.837 at four retrains

TABLE I

POLICY COMPARISON ON CESNET-TLS-YEAR22, DEFAULT $r = 10$. MEAN OVER FIVE SEEDS; ± IS THE STANDARD DEVIATION OF MACRO-F1. GAP: POINTS OF MACRO-F1 BELOW THE DP-OPTIMAL SCHEDULE WITH THE SAME NUMBER OF RETRAINS (MEAN OVER 5 SEEDS). COST: UNITS OF ONE RETRAIN, ERROR COST CLIPPED AT ZERO. DELAY IN WEEKS; FT: FALSE-TRIGGER RATE.

	Policy	Retrains	Macro-F1	Gap	Cost	Delay	FT
P0	Never	0.0	0.709 ± 0.002	-0.00	101.4	–	–
P1	Periodic, k=4	12.0	0.855 ± 0.001	0.46	16.6	0.6	0.50
P2	Periodic, k=1	49.0	0.868 ± 0.000	0.01	50.0	0.0	0.92
–	Periodic, k=8	6.0	0.841 ± 0.000	0.58	15.5	2.7	0.00
P3	Threshold on D	3.4	0.825 ± 0.006	0.80	19.6	5.8	0.00
P4	Myopic cost rule	4.0	0.831 ± 0.001	0.61	16.0	3.2	0.00
P4c	Cumulative cost rule	5.4	0.842 ± 0.002	0.26	11.3	2.1	0.00
–	Cumulative, r=100	11.0	0.853 ± 0.002	0.46	15.0	0.8	0.41
P5	Threshold oracle	6.8	0.846 ± 0.001	0.30	12.0	1.4	0.00

TABLE II

CORRELATION OF EACH ORIENTED SIGNAL WITH TRUE WEEKLY MACRO-F1 LOSS OF THE NEVER-RETRAINED MODEL (50 WEEKS).

Signal	Family	Spearman	Pearson
Max. probability (mean)	Confidence	0.70	0.87
Max. probability (10th pct.)	Confidence	0.72	0.87
Top-2 margin (mean)	Confidence	0.72	0.88
Entropy (mean)	Confidence	0.67	0.86
JS divergence, predicted classes	Divergence	0.05	0.32
KS statistic (mean over features)	Input drift	0.20	0.78
MMD^2	Input drift	0.17	0.74
Reject rate	Reject	0.73	0.89
Composite D	All	0.58	0.86

TABLE III

COST-RATIO SWEEP (FIVE-SEED MEANS). RETRAINS, MACRO-F1 AND GAP IN POINTS.

	P4 (myopic)			P4c (cumulative)		
r	Ret.	F1	Gap	Ret.	F1	Gap
0.3	0.0	0.709	-0.00	1.8	0.751	5.54
1	0.0	0.709	-0.00	2.8	0.805	2.26
3	0.0	0.709	-0.00	3.0	0.828	0.26
10	4.0	0.831	0.61	5.4	0.842	0.26
30	5.4	0.840	0.43	8.4	0.849	0.46
100	6.2	0.842	0.51	11.0	0.853	0.46
300	6.4	0.842	0.55	12.6	0.856	0.36
1000	6.4	0.842	0.55	13.8	0.857	0.34

TABLE IV

CESNET-QUIC22: RETRAIN DAYS BY SEED (S0--S4). THE DOCUMENTED CERTIFICATE CHANGE IS ON DAY 22.

	S0	S1	S2	S3	S4
P3 threshold	–	–	–	–	–
P4 myopic	22	22	22	22	22
P4c cumulative	22	22	22	22	22
P5 oracle	10, 18	12	9, 14	10, 16	10, 15
Change point in D	12	11	11	12	11

and reaches 0.859 at twelve, so later retrains buy little. Every adaptive family and periodic retraining lie close to it, and none dominates. Weekly retraining gains 1.2 points over P4c at r = 300 (12.6 retrains, macro-F1 0.856) for nearly four times the retrains. The two rules respond very differently to the knob (Table III). At r = 3 P4 never fires and stays at the macro-F1 of never retraining, while P4c with the same g retrains 3.0 times; at high r P4 saturates near 6.4 retrains while P4c keeps adapting. Inertness at low r and saturation at high r are the behaviors the cumulative rule was built to remove. In cost, measured as regret to the cost-optimal schedule at r = 10 (mean over 5 seeds), P4c is 1.7 retrain units above the optimum, P4 6.3, P3 10.0, never retraining 91.8, and the best periodic interval, chosen in hindsight, 5.8. P4c has the lowest regret of these policies only for $3 \le r \le 100$: at r = 300 the hindsight-tuned calendar does better (37 against 74), which fits the under-pricing of late-year loss by the frozen g (Sec. VII).

D. Timing on TLS

Fig. 3 shows the mechanism for seed 0. D moves from 1.8 to 3.2 to 4.6 over weeks 9–11 as the exporter update takes hold; P4c retrains at the end of week 10 and weekly F1 recovers from 0.78 to 0.85. It retrains at weeks 13, 25, 32, 38 and 47 afterward. Week 25 is a one-week dip that even weekly retraining cannot avoid, since a model cannot be retrained on data it has not yet seen.

E. Cross-protocol validation on QUIC

In all five seeds P4 and P4c retrain on day 22, the day of the documented certificate change, with no earlier retrain and zero detection delay, at the cost of one retrain; P3 never retrains (Table IV). The periodic schedules retrain on days 11, 15, 19, 23 and 27 (every 4 days) or days 14 and 21 (every 7 days). The first brings a model that has seen the change one day after P4c does, after five retrains instead of one; the second retrains on day 21, a day too early to include any post-change data. Mean macro-F1 over the replay is 0.666 for P4, 0.666 for P4c, 0.684 for P5 and 0.651 for never retraining (Fig. 4).

The picture is less clean than a single step. The oracle retrains earlier, on days 9–18 depending on the seed, because the model had already lost accuracy before the change: never-retrained F1 falls from 0.68 on day 8 to 0.63 on day 21 (seed 0), and a change point appears at day 11 or 12 in every seed. The simple F1 gap across day 22 is therefore small

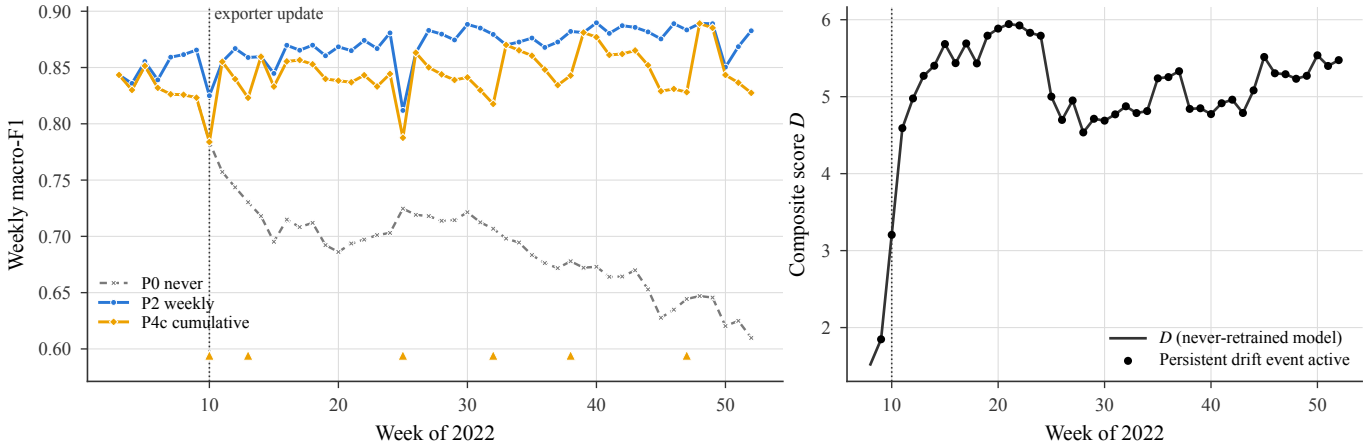


Fig. 3. CESNET-TLS-Year22, seed 0. Left: weekly macro-F1 for never retraining, weekly retraining and P4c; triangles mark P4c's retrains. Right: the composite score D of the never-retrained model; dots mark weeks with an active persistent drift event. The dotted line is the week-10 exporter update.

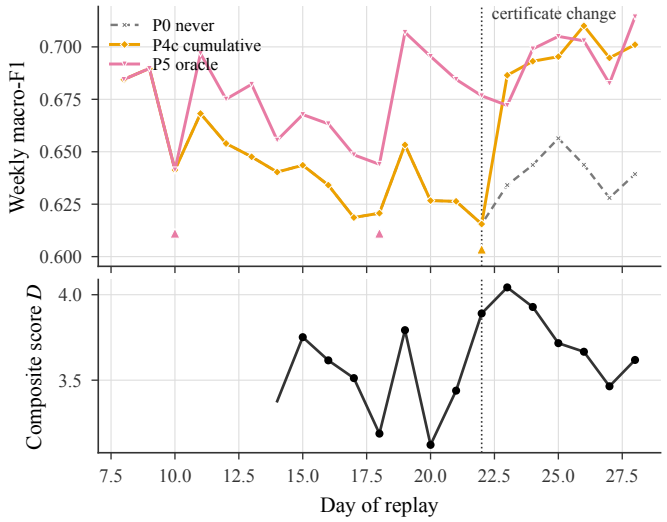


Fig. 4. CESNET-QUIC22, seed 0 (decisions are daily). Top: macro-F1 of never retraining, P4c and the oracle; triangles mark retrains. Bottom: composite score D of the never-retrained model. The dotted line is the documented certificate change on day 22.

and noisy (mean 0.004, range −0.001 to 0.009), and we do not offer it as evidence; the timing match is. Two caveats apply. The five seeds share one physical event and one data sample, so 5/5 measures stability to classifier randomness, not five independent confirmations. And this check exposed a configuration-merge bug: the first run inherited the TLS exporter-event day (10) as the QUIC event day, which left decisions unchanged but produced an artificial 12-day delay and a pre/post gap of 0.043. Both are corrected above.

VII. Discussion and Limitations

Labels are not eliminated, only rationed. The loss map g needs eight labeled weeks, which here include the exporter update, and a retrain needs two. In the late window (weeks 40–45), g predicts a mean loss of 0.104 where the never-retrained model's true loss is 0.188: it ranks days correctly (Spearman 0.77) but under-predicts the level by 0.084, so a map frozen in March underprices drift in November and biases both cost rules toward waiting. Recalibration is an obvious extension we did not test. The composite is carried by the confidence family, which is cheap to compute, and a confidence-only score would correlate better with loss than the composite does. The cost constants are illustrative and the fixed W0 reference makes absolute cost depend on the reference choice, which is why we lean on macro-F1 against retrains and on regret to the DP schedule. The experiments use sampled flows from one vantage point, a closed world of 50 services and weekly (TLS) or daily (QUIC) decisions; QUIC22 offers four weeks and one documented event. We have not studied partial retraining [28], label-budgeted retraining [24] or open-world traffic.

VIII. Conclusion

A frozen encrypted-traffic classifier falls from 0.840 to 0.610 macro-F1 over a year, and a monitor that sees only its outputs and inputs can tell when. Our drift score correlates with true loss without reading a label, and a cumulative-cost rule converts it into retraining decisions that land near the best schedule hindsight allows, reach it without a tuned calendar, and fire on the day of a documented QUIC change that a fixed threshold misses. Calendars tuned in hindsight remain competitive at matched budgets, and most of the score's signal comes from classifier confidence. The pipeline, the label-isolation tests and the exact-optimum harness are at `github.com/Darsh2905/Label-free-retraining-triage`, so these comparisons can be repeated on other traffic and other drifts.

References

[1] Y. D. Goli and R. Ambika, "Network traffic classification techniques—a review," in *Proc. Int. Conf. Comput. Techn., Electron. Mech. Syst. (CTEMS)*, 2018, pp. 219–222.

[2] F. Al-Obaidy, S. Momtahen, M. F. Hossain, and F. Mohammadi, "Encrypted traffic classification based ML for identifying different social media applications," in *Proc. IEEE Can. Conf. Elect. Comput. Eng. (CCECE)*, 2019, pp. 1–5.

[3] S. M. Rachmawati, D.-S. Kim, and J.-M. Lee, "Machine learning algorithm in network traffic classification," in *Proc. Int. Conf. Inf. Commun. Technol. Converg. (ICTC)*, 2021, pp. 1010–1013.

[4] J. Luxemburk, K. Hynek, and T. Čejka, "Encrypted traffic classification: the QUIC case," in *Proc. 7th Netw. Traffic Meas. Anal. Conf. (TMA)*, 2023, pp. 1–10.

[5] D. Shamsimukhametov, A. Kurapov, M. Liubogoshchev, and E. Khorov, "Is encrypted ClientHello a challenge for traffic classification?" *IEEE Access*, vol. 10, pp. 77 883–77 897, 2022.

[6] V. Gudepu, V. R. Chintapalli, P. Castoldi, L. Valcarenghi, B. R. Tamma, and K. Kondepu, "Adaptive retraining of AI/ML model for beyond 5G networks: A predictive approach," in *Proc. IEEE 9th Int. Conf. Netw. Softwarization (NetSoft)*, 2023, pp. 282–286.

[7] H. Kanda, H. Okamura, and T. Dohi, "A note on optimal retraining strategy for ML systems," in *Proc. 10th Int. Conf. Dependable Syst. Their Appl. (DSA)*, 2023, pp. 730–733.

[8] A. Kurapov, D. Shamsimukhametov, M. Liubogoshchev, and E. Khorov, "LTI: Encrypted traffic classification framework considering data drift," in *Proc. IEEE Int. Black Sea Conf. Commun. Netw. (BlackSeaCom)*, 2024, pp. 352–355.

[9] L. Jančička, J. Koumar, D. Soukup, and T. Čejka, "Analysis of statistical distribution changes of input features in network traffic classification domain," in *Proc. IEEE Netw. Oper. Manage. Symp. (NOMS)*, 2024, pp. 1–4.

[10] N. Malekghaini, E. Akbari, M. A. Salahuddin, N. Limam, R. Boutaba, B. Mathieu, S. Moteau, and S. Tuffin, "Deep learning for encrypted traffic classification in the face of data drift: An empirical study," *Comput. Netw.*, vol. 225, 2023, art. no. 109648.

[11] B. Anderson and D. McGrew, "Identifying encrypted malware traffic with contextual flow data," in *Proc. ACM Workshop Artif. Intell. Secur. (AISec)*, 2016, pp. 35–46.

[12] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf. (WWW)*, 2022, pp. 633–642.

[13] H. Zhang, L. Yu, X. Xiao, Q. Li, F. Mercaldo, X. Luo, and Q. Liu, "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf. (WWW)*, 2023, pp. 2066–2075.

[14] S. Yang, X. Zheng, J. Li, J. Xu, and E. C. H. Ngai, "Multi-scale contrastive attention representation learning for encrypted traffic classification," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2024, pp. 4173–4177.

[15] I. Akbari, M. A. Salahuddin, L. Aniva, N. Limam, R. Boutaba, B. Mathieu, S. Moteau, and S. Tuffin, "A look behind the curtain: Traffic classification in an increasingly encrypted web," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 5, no. 1, pp. 1–26, 2021.

[16] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2017, pp. 1723–1732.

[17] W. Yang, R. Su, Y. Cheng, and J. Guo, "A concept drift detection approach based on Jensen–Shannon divergence for network traffic classification," in *Proc. 5th Int. Conf. Artif. Intell. Pattern Recognit. (AIPR)*, 2022, pp. 982–987.

[18] E. Lughofer, E. Weigl, W. Heidl, C. Eitzinger, and T. Radauer, "Recognizing input space and target concept drifts in data streams with scarcely labeled and unlabelled instances," *Inf. Sci.*, vol. 355–356, pp. 127–151, 2016.

[19] T. S. Sethi and M. Kantardzic, "On the reliable detection of concept drift from streaming unlabeled data," *Expert Syst. Appl.*, vol. 82, pp. 77–99, 2017.

[20] V. M. A. Souza, F. A. Chowdhury, and A. Mueen, "Unsupervised drift detection on high-speed data streams," in *Proc. IEEE Int. Conf. Big Data*, 2020, pp. 102–111.

[21] S. Aminikhanghahi, T. Wang, and D. J. Cook, "Real-time change point detection with application to smart home time series data," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 5, pp. 1010–1023, May 2019.

[22] A. N. Smirnov, "The choice of the cost parameter in the PELT method for determining the change points for time series parameters in the hidden channel model," in *Proc. 5th Int. Youth Conf. Radio Electron., Elect. Power Eng. (REEPE)*, 2023, pp. 1–5.

[23] S. S. Bagui, M. P. Khan, C. Valmyr, S. C. Bagui, and D. Mink, "Model retraining upon concept drift detection in network traffic big data," *Future Internet*, vol. 17, no. 8, 2025, art. no. 328.

[24] W. Liu, C. Zhu, Z. Ding, H. Zhang, and Q. Liu, "Multiclass imbalanced and concept drift network traffic classification framework based on online active learning," *Eng. Appl. Artif. Intell.*, vol. 117, 2023, art. no. 105607.

[25] W. Cai, C. Hou, M. Cui, B. Wang, G. Xiong, and G. Gou, "Incremental encrypted traffic classification via contrastive prototype networks," *Comput. Netw.*, vol. 250, 2024, art. no. 110591.

[26] S. Liu, F. Bronzino, P. Schmitt, A. N. Bhagoji, N. Feamster, H. G. Crespo, T. Coyle, and B. Ward, "LEAF: Navigating concept drift in cellular networks," *Proc. ACM Netw.*, vol. 1, no. CoNEXT2, pp. 1–24, 2023.

[27] J. Li, H. Yu, Z. Zhang, X. Luo, and S. Xie, "Concept drift adaptation by exploiting drift type," *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 4, pp. 1–22, 2024.

[28] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, 2014.

[29] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, "A kernel two-sample test," *J. Mach. Learn. Res.*, vol. 13, pp. 723–773, 2012.

[30] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on loess," *J. Off. Stat.*, vol. 6, no. 1, pp. 3–73, 1990.

[31] R. Killick, P. Fearnhead, and I. A. Eckley, "Optimal detection of changepoints with a linear computational cost," *J. Amer. Statist. Assoc.*, vol. 107, no. 500, pp. 1590–1598, 2012.

[32] K. Hynek, J. Luxemburk, J. Pešek, T. Čejka, and P. Šiška, "CESNET-TLS-Year22: A year-spanning TLS network traffic dataset from backbone lines," *Sci. Data*, vol. 11, 2024, art. no. 1156.

[33] J. Luxemburk, K. Hynek, T. Čejka, A. Lukačovič, and P. Šiška, "CESNET-QUIC22: A large one-month QUIC network traffic dataset from backbone lines," *Data Brief*, vol. 46, 2023, art. no. 108888.

[34] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 3146–3154.